

Evaluación Globalizadora B Sistemas Operativos de Propósito General

Bustamante, Fernando

fernando.miguel.bustamante@gmail.com

Portas, Nahuel

nahuluciano@gmail.com

UNIVERSIDAD TECNOLÓGICA NACIONAL-FACULTAD REGIONAL AVELLANEDA

Sumario

1. Fundamentación.....	1
2. Objetivos.....	2
2.1 Arquitectura de sistema.....	2
3. Alcance.....	3
4. Diagrama de software.....	3
5. Diagrama de tareas de EGA.....	3
6. Diagrama de conexiones completo.....	3
7. Listado de comandos implementados.....	3
8. Conclusiones.....	3
9. Referencias.....	4
ANEXO 1: Diagrama de software.....	5
ANEXO 2: Diagrama de tareas.....	6
ANEXO 3: Diagrama de conexiones.....	7

Abstract

Este proyecto implementa un sistema de comunicación serie entre dos sistemas operativos de diferentes dominios; siendo uno de ellos un RTOS orientados a sistemas embebidos y el otro un sistema operativo de propósito general (GPOS), basado en Linux. La arquitectura propuesta utiliza una Raspberry Pi Pico ejecutando FreeRTOS como plataforma embebida y una Raspberry Pi 4 con Linux como sistema de propósito general. El núcleo de la implementación consiste en el desarrollo de un módulo kernel personalizado para el GPOS, permitiendo la comunicación UART de bajo nivel entre ambos sistemas. El proyecto demuestra la viabilidad de

integrar sistemas heterogéneos mediante el desarrollo de drivers específicos, requiriendo conocimientos avanzados de programación del kernel de Linux para su implementación.

Palabras clave: RTOS, GPOS, FreeRTOS, Linux kernel, UART, Raspberry Pi Pico, Raspberry Pi 4, Comunicación Serie, desarrollo de driver.

1. Fundamentación

Los sistemas embebidos modernos presentan una creciente complejidad, si bien puedan capturar datos de diferentes sensores, no posee una capacidad de procesamiento y de recursos tan elevada como un ordenador convencional. Para mejorar la fiabilidad de

los datos y disponer de mayor potencia de procesamiento se implementan comunicaciones entre sistemas heterogéneos, donde el GPOS aporta su gran capacidad de procesamiento, mientras que el RTOS aporta su capacidad de ejecución en alcionio que demandan respuestas en tiempo estricto.

La comunicación fiables entre estos dominios diferentes representa un desafío. En este proyecto se encara dicho desafío mediante el desarrollo de un driver personalizado en el kernel de Linux, lo cual permite una comunicación de bajo nivel entre la Raspberry Pi Pico y la Raspberry Pi 4. Este enfoque garantiza un control preciso sobre el hardware y optimiza el rendimiento, evitando las limitaciones de las soluciones en espacio de usuario.

La razón de usar la plataforma Raspberry es por la compatibilidad entre los pines de ambas placas, ya que ambas manejan los mismos niveles de tensión de 3.3V. Aunque es posible usar dos plataformas diferentes se debe prestar atención a los niveles de tensión que manejan los pines y, si es necesario, habría que adaptar los niveles de tensión.

El desarrollo de un driver personalizado garantizara un mayor rendimiento, ya que estamos aprovechado las ventajas de ambos sistemas operativos, viendo al conjunto RTOS + Pico como una caja negra. Pero ademas podemos gestionar la información y controlar de manera mas precisa el acceso al hardware por parte de una aplicación de usuario. Este driver permitirá que una aplicación de usuario lo pueda usar como intermediario de manera que la aplicación de usuario accede al recurso de forma gestionada y precisa, sin la necesidad de que sea la propia aplicación la acceda de manera directa.

2. Objetivos

Como objetivo general se desarrollara un sistema de comunicación confiable entre Raspberry Pi Pico y Raspberry Pi 4, mediante una implementación de un driver UART personalizado en el kernel de Linux.

Para cumplir el objetivo general se debieron de cumplir los siguientes objetivos:

- Creación de una interfaz grafica para interactuar con el driver
- Creación del driver personalizado, que se encargara de enviar la información por el hardware

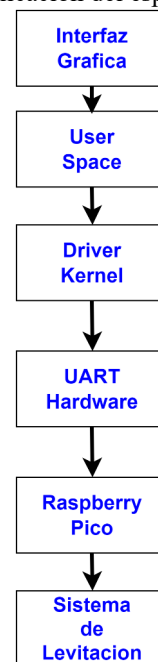
-Uso de la Raspberry Pi Pico con un código en FreeRTOS previamente cargado y con los necesario para recibir por UART, esta parte seria como una caja negra.

2.1 Arquitectura de sistema

La Raspberry Pi Pico tiene cargado un código en FreeRTOS, el cual consiste en mantener en levitación una esfera de tergopol. Como el código es funcional solo se agregara la tarea necesaria para recibir por UART la información necesaria para la configuración.

En la Raspberry Pi 4 podemos dividir en dos partes; el espacio de usuario y el espacio de kernel. En el primero se hará uso de una interfaz grafica que permita el ingreso de los datos a configurar. Esta parte se encaro creado una pagina web de manera que se pueda conectar desde cualquier lugar y configurar sus parámetros.

En la lado del espacio de kernel actuara el driver personalizado el cual enviara por UART la información de la aplicación del espacio de usuario.



3. Alcance

Si bien la creación de un driver personalizado garantiza un mayor rendimiento, también es vital conocer que existe una fuerte limitación, por el hecho que nuestro driver estará limitado al hardware de la Raspberry Pi 4. Pero incluso dentro de la propia Raspberry Pi 4 debería considerarse que al intentar usar uno de los UART (UART1 o UART0) requerirá desactivar la función de Bluetooth, la cual podría ser usada por un teclado o ratón.

4. Diagrama de software

En el ANEXO 1 se puede ver el diagrama de software. El mismo está dividido en dos partes: la Raspberry Pi 4 y la Raspberry Pi Pico.

5. Diagrama de tareas de EGA

En el ANEXO 2 se puede ver el diagrama de tareas. En este caso se modificó la EGA, de manera que se adicionó una tarea para capturar los datos del UART, mientras que la tarea *task_setpoint* se mantiene pero quedará comentada. El resto de las tareas permanece sin cambios.

6. Diagrama de conexiones completo

En el ANEXO 3 se puede ver el diagrama de conexiones. En este diagrama se verán las conexiones físicas del conjunto Raspberry Pi 4 y Raspberry Pi Pico.

7. Listado de comandos implementados

Los comandos implementados son los datos que se envían desde la aplicación web hacia la Pico. Para ser más claros, insertamos los datos en la interfaz gráfica, en la misma aplicación, los datos se ordenan y procesan para luego enviarlos al driver y que este los envíe a la Pico. La forma del comando es básicamente una cadena de caracteres que dependerá de cada EGA.

Para nuestro caso se pretende enviar un comando el cual indique: el nivel de flotación (SP), la altura máxima para control (SM) y la altura mínima de

control (Sm). Tenemos dos formas enviar un comando a la vez, o enviar todo en una sola cadena. Se optó por la segunda forma, para lo cual el comando que enviaremos y que deberá ser recibido por la Pico tiene el siguiente formato:

SP:valor1,SM:valor2,Sm:valor3

La cadena es un único comando que separa los datos usando “,” para facilitar el procesamiento del lado del RTOS.

8. Conclusiones

La implementación de este proyecto confirmó la viabilidad de integrar sistemas operativos heterogéneos mediante el desarrollo de software de bajo nivel. Se logró:

- La integración de dos sistemas operativos diferentes.

- Desarrollo exitoso del módulo kernel controlador personalizado.

- Comando funcional, se validó el protocolo de comando serial emitiendo el envío atómico de todos los parámetros en una sola transacción de datos.

Si bien los objetivos se lograron estos presentaron implicaciones más duras para el aprendizaje:

- Curva de aprendizaje más lenta, ya que el desarrollo del módulo kernel de Linux implica un profundo conocimiento y entendimiento de la arquitectura de Raspberry Pi 4, así como del kernel de Linux.

- Dificultad en el mantenimiento y la portabilidad, debido al alto acoplamiento del driver al hardware lo cual obliga a realizar una reimplantación significativa.

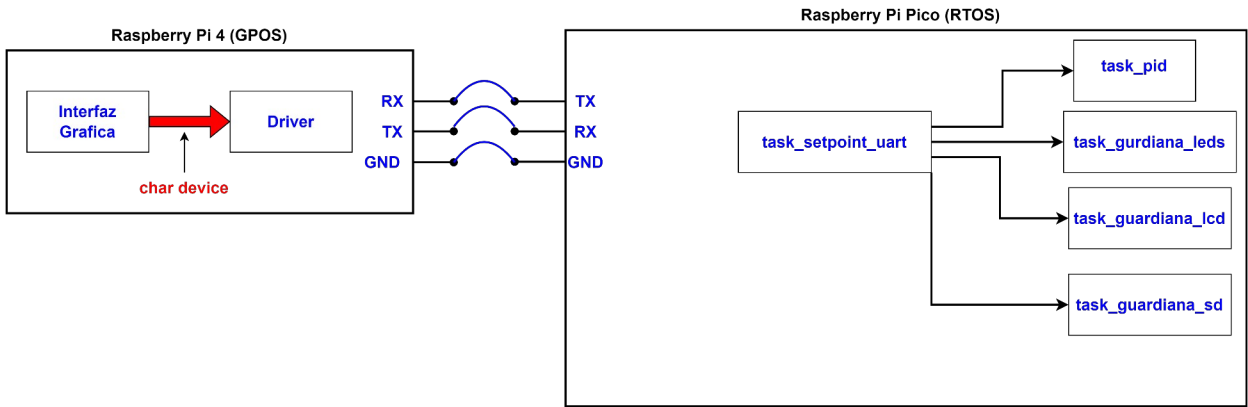
- Considerar la versión del kernel a utilizar, este punto debe chequearse ya que hay versiones que no manipulan ciertas APIs del kernel o simplemente se las actualiza.

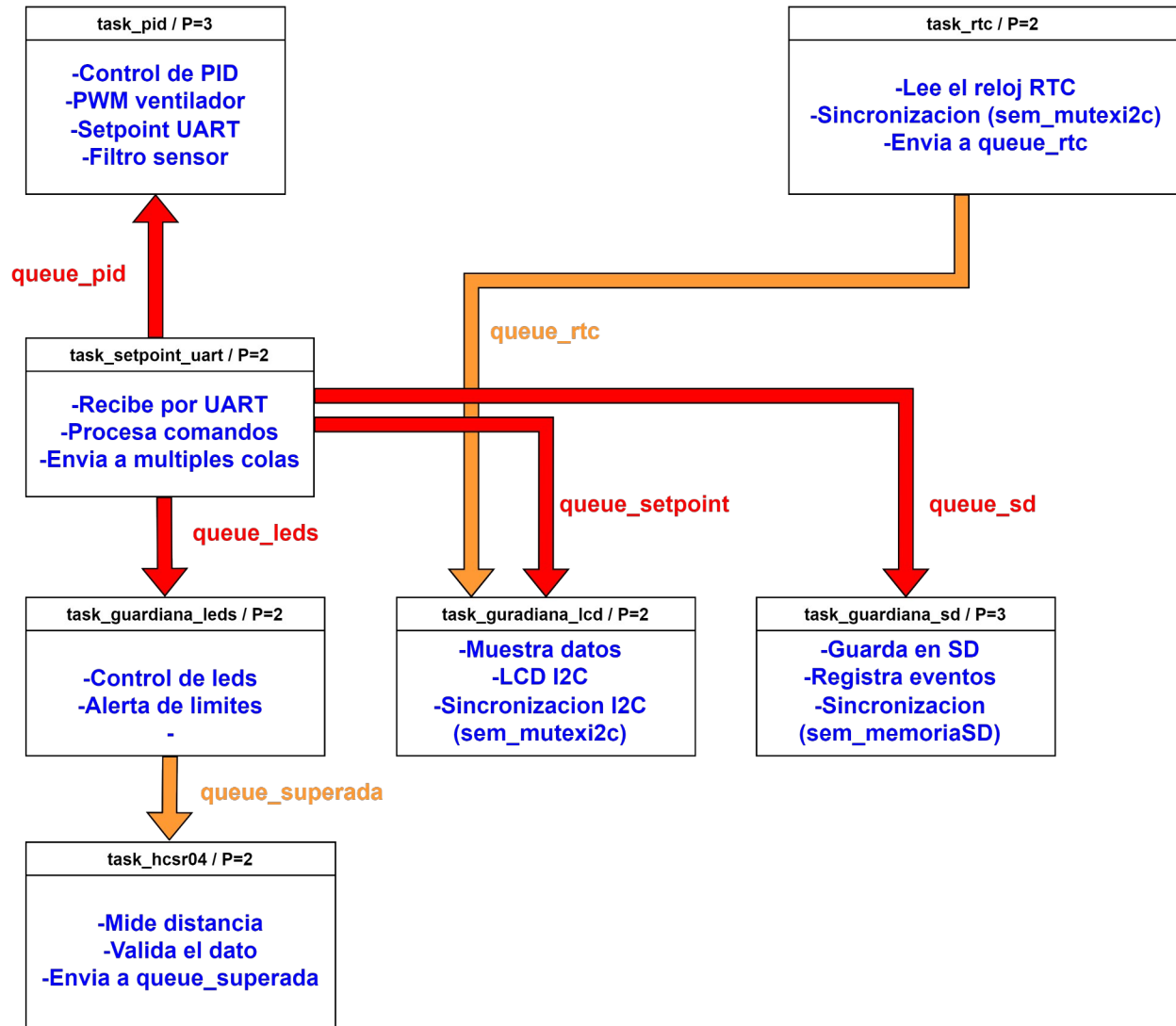
- El desarrollo de este proyecto implicó trabajar a nivel de hardware lo que dificulta el desarrollo y su portabilidad a otras plataformas.

9. Referencias

[1] Peter Jar Salzman, Michael Buriam, Ori Pomerantz, Bob Motram, The Linux Kernel Module: Programming Guide.
<https://sysprog21.github.io/lkmpg/#what-is-a-kernel-module>

ANEXO 1: Diagrama de software



ANEXO 2: Diagrama de tareas

ANEXO 3: Diagrama de conexiones